NN&DL

Neural Networks: Architecture and Training

S. Coniglio

Dipartimento di Scienze Economiche, UniBg

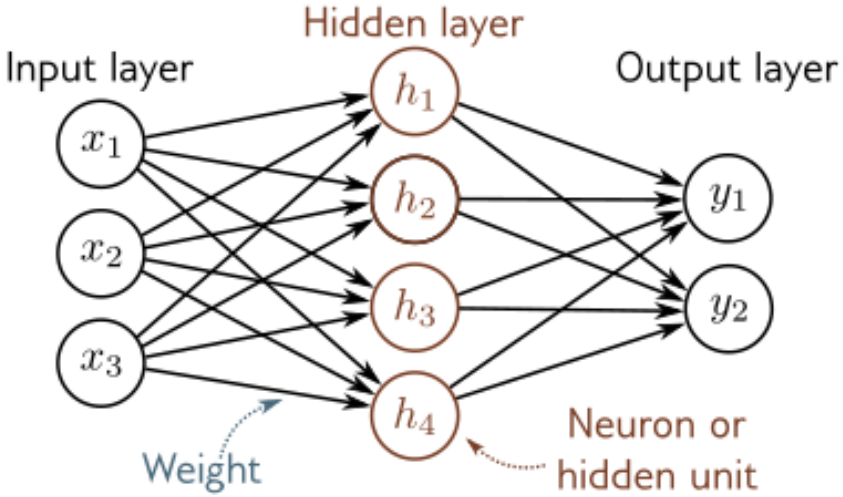
Sem. II

Summary

1. The architecture of neural networks

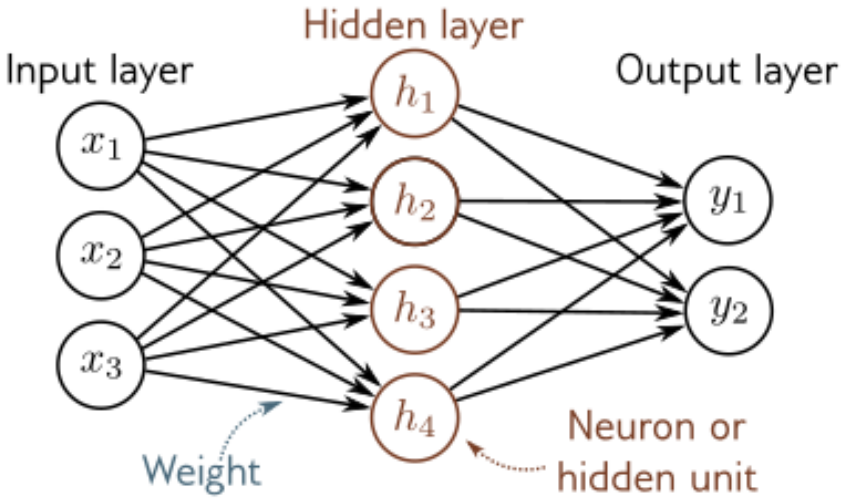
Multi-layer neural networks

NNs with more than a single neuron are organized over **layers**, including **hidden layers** that lie between the input and output layers (they are called *hidden* because, at inference time, their internal computations are not directly visible to the user).



The specific arrangement of layers and connections in a multi-layer network is referred to as its **architecture**.

Traditionally, a NNs with at least one hidden layer is called a **multi-layer perceptron** (MLP) and, also traditionally, MLPs are NNs of the same architecture as the one in the figure (see further).

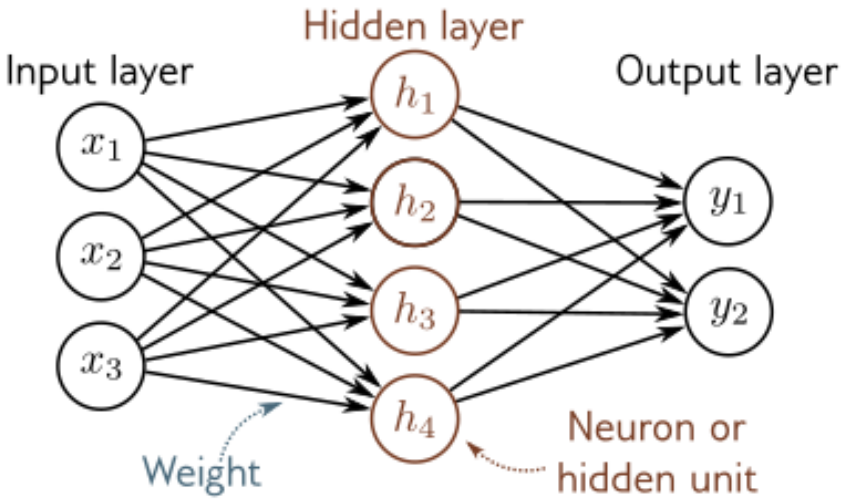


The **input layer** contains as many neurons as the dimension n_{in} of the input signal to the NN. These are **dummy neurons** (i.e., they are not *computational units*), as they only propagate the input signal to the output—their activation function is the **identity function**, i.e., $\Phi : x \mapsto x$, for any input $x \in \mathbb{R}$.

If, e.g., the NN is designed to process 5-dimensional real-valued vectors, the input layer will feature $n_{in} = 5$ neurons.

The NN is a **single-layer network** when the input(s) are directly connected to the output(s) without any intermediate layer.

The NN is a **multi-layer network** whenever one or more "intermediate" layers are present.

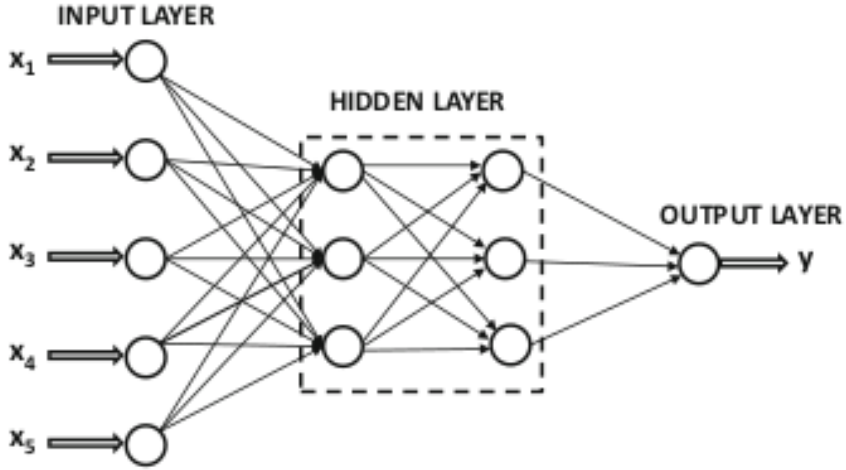


The **output layer** contains as many neurons as the dimension n_o of the output signal the NN should produce.

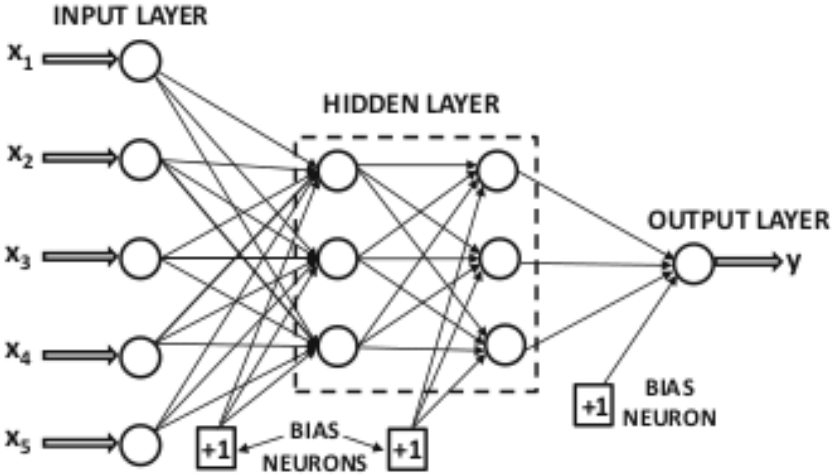
A network tasked with solving a binary classification problem will have a single output neuron ($n_o = 1$), often outputting one of two values: $0, 1$ or $-1, 1$. A network tasked with solving a more complex problem, such as forecasting the value of a multi-variate phenomenon, will have more. A transformer, for instance, outputs a vector of probabilities with as many components as the tokens in the dictionary (we'll see this).

The layers that connect the input layer to the output layer are called **hidden layers**.

Just as in a single-node network, **bias neurons** (used for, e.g., shifting the firing threshold of a step/signum activation function) can (and should!) be incorporated into both the hidden layers and the output layer of a multi-layer network.



(a) No bias neurons



(b) With bias neurons

We won't explicitly indicate the bias anymore in the future.

The behavior of a NN depends on two factors:

- the **architecture** of the NN, i.e., the number of neurons it contains and the way they are interconnected over the layers; this is often called **model**
- the synaptic **weights** (and, *biases*), which are, ultimately, determined by two factors:
  - the training data
  - the training algorithm.

Quantities such as the number of neurons per layer and the number of layers are (among others) often called **hyperparameters** of the NN, differently from the NN's weights, which are called **parameters**.

Training a NN amounts to choosing its weights (and biases).

The choice of the right architecture is often called **architecture search**—it's not a well-solved problem yet and is often carried out with heavily heuristic, "bag-of-trick" methods.

Two common architectures

Feed-Forward NNs

These are NNs in which the connections form an acyclic graph (i.e., no connections go back from a neuron on the right to a neuron on the left). In a Feed-Forward NN, the data (and the computations) flow sequentially from the input (layer) to the output (layer) through the hidden layers.

Fully-Connected NNs

These are NNs where every node in one layer connects to every node in the next layer. Fully-connected NNs are, traditionally, the most common type of NN (or they were before Deep Learning bloomed). In the fully-connected case, the NN's architecture is solely determined by:

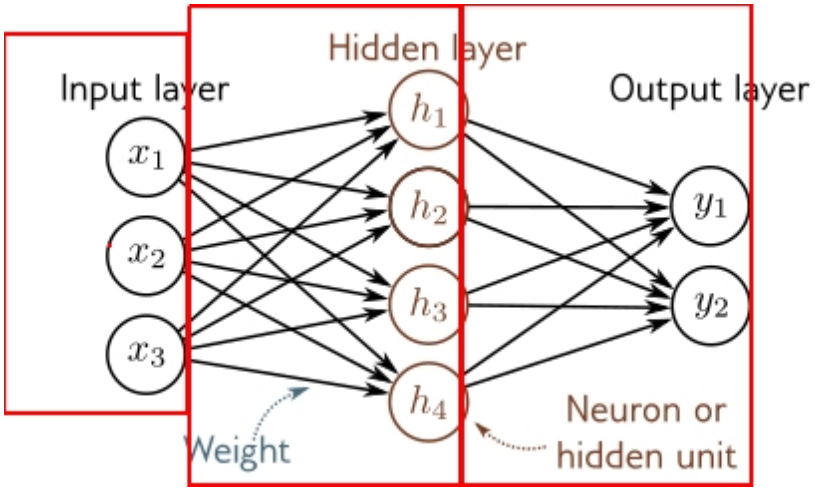
- The number of layers.
- The number (and type) of neurons in each layer.

Typically, we refer to a feed-forward, fully-connected NNs as an MLP (most names in AI are just wrong...).

Oftentimes, we'll picture a NN just as a collection of layers, where a **layer** is contains

- its neurons
- (their biases)
- the synaptic weights that connect its *previous* layer to it.

Thus, when counting layer we'll (often!) ignore the input layer (in line with what we did wit the Mark I Perceptron, where we looked at each R-unit in isolation). This is because the input layer only transmits data without carrying out any computations (its neurons are *dummy* neurons).



This NN has 2 (not 3) layers.

2. Training neural networks

Machine learning

Quoting Tom M. Mitchell (founder of the ML department at CMU), we can define **Machine Learning** as follows:

> *A* **computer program** *is said to learn from experience* E *with respect to some class of tasks* T *and performance measure* P *if its performance at tasks in* T*, as measured by* P*, improves with experience* E*.*

This is a reasonable definition, albeit it applies to an **iterative ML algorithm**, whose performance improves as experience is accumulated.

It'd be better to define a (learning) *problem* whose solution method will be an *algorithm*. The latter *can* be iterative, but it doesn't necessarily have to—there are cases where the learning algorithm does not evolve a solutions that improves over time.

> **Learning problem**: *given experience E and some class of tasks T and performance measure P, find a model M which achieves maximum performance at tasks in T, as measured by P.*

Such a performance P should be non-decreasing in E.

We call **training** the act of finding model M—where, typically, M is parametric, its family is fixed. and we "just" look for the best choice of parameters for it.

Training a NN vs. inference with a NN

In the context of NNs, training the act of deciding the numerical value of its weights (and biases).

The NN's architecture, or model, is typically fixed *a-priori*, albeit there are a number of techniques of **model selection** that can be used to identify the "right" architecture.

Once the network has been trained (i.e., its weights have been fixed), we can use it in **inference** to make predictions (or to generate content, if we are working with generative NNs such as transformers trained for NLP tasks).

This is in line with what we'd do with any mathematical model. Indeed, the main task of a NN (or of any ML model) is to extrapolate the input-output relationship that's hidden within the training data from beyond the set of examples contained in it in order to be able to apply the same relationship to previously unseen inputs to **infer** (hence "infernece") their outputs. We call such a capability **generalization capability**.

(Transformers (the NNs behind LLMs) are not too different: they extrapolate the next-word (or token) relationship between a chunk of text and the next (missing, or *masked*) word).

Learning (in the supervised-learning setting)

Let's assume that we are interested in learning some hidden phenomenon which maps an independent variable x to a dependent variable y.

Following again the FNA example, let $x \in \mathbb{R}^{n_{in}}$ be an n_{in}-dimensional real-valued vector representing various properties of a sample, and $y \in \{-1, 1\}$ be a binary value telling us whether the sample was taken from a benign (-1) or malignant (1) tumor—the output dimension is $n_{out} = 1$.

We assume the unknown mapping is realized by a function $\rho : x \mapsto y$, which is completely unknown to us.

We call the components of x **features**; x itself is called a **feature vector**.

We call y the **target** variable or **label** (or, sometimes, **ground truth**).

In a classical machine learning setting, our task is two-fold:

A) we should determine by which **parametric model** f we wish to approximate the hidden function ρ
B) we should determine the numerical value of f's parameters.

Let us call w the vector containing the parameters of f. For instance, if $f(x) = ax^2 + bx + c$, w is the vector of parameters $w = (a, b, c)^\top$.

To make the dependence of f on w clear, going forward we will adopt either the notation f_w or the notation $f(x; w)$.

Distance functionals

In principle, looking for the best approximation of ρ calls for solving is the following **optimization problem**:

$$\min_{w} \text{distance}(f_w, \rho),$$

where distance is a **distance functional** (i.e., a function of functions).

The distance between two functions can be measured in a number of ways.

Assuming that both f and ρ are continuous and enjoy some integrability properties, the most intuitive way is to take the square of the integral of their difference:

$$\int_D \Big(f(x) - \rho(x)\Big)^2 dx,$$

where D is the domain of both functions (such a quantity is the *squared Lebesgue distance* of two functions). If f and ρ have a finite domain rather than a continuous one, the previous integral boils down to computing

$$\sum_{x \in D} \Big(f(x) - \rho(x)\Big)^2,$$

which, once divided by $|D|$, coincides with the the well-known **mean squared error** or **MSE**.

Statistical learning approach

Independently of the choice of a distance functional or another, this problem

$$\min_{w} \text{distance}(f_w, \rho),$$

is clearly unsolvable. This is because we have no access to ρ. If we did, we'd be facing an approximation problem (rather than a learning one), but this is not the case as <u>we do not have access to ρ</u>).

The way we sort this issue is by following the classical principle of **statistical learning**. We assume that ρ's structure can be inferred in a sufficiently precise manner if we are given sufficiently many (let's say k) input-output **examples** or **samples** or **observations** of type (x_i, y_i) with $y_i = \rho(x_i)$, one per index $i \in [k]$.

We call $\{(x_i, y_i)\}_{i \in [k]}$, i.e., the collection of such examples, **training set**.

Loss function

We formalize the statistical learning problem by introducing a **loss function**, a function L which quantifies the distance between f_w and ρ on the examples we collected in the training set.

A possible choice for L could be:

$$L\left(\{(x_i, y_i)\}_{i\in[k]}, w\right) = \frac{1}{k}\sum_{i\in[k]}\Big(\underbrace{f_w(x_i)}_{\hat{y}_i} - \underbrace{\rho(x_i)}_{y_i}\Big)^2,$$

which is, again, the well-known **mean squared error** (MSE) (notice that, here, it is calculatd on the subset of D that we observed) or maybe

$$L\left(\{(x_i, y_i)\}_{i\in[k]}, w\right) = \frac{1}{k}\sqrt{\sum_{i\in[k]}\Big(\underbrace{f_w(x_i)}_{\hat{y}_i} - \underbrace{\rho(x_i)}_{y_i}\Big)^2},$$

which is the equivalently well-known **root mean squared error** (RMSE).

w is found by calculating the minimum of the loss function by relying on *some* **mathematical optimization algorithm** (typically, but not necessarily, of iterative type—i.e., an algorithm that iteratively updates w until a good enough value is found).

3. Paradigms of learning

Supervised learning

Supervised learning (SL) is the type of learning problem we talked about so far.

In it, the labels $y_i, i \in [k]$, provide us with a form of **supervision** that guides the learning process by minimizing the **learning errors** $|f_w(x_i) - y_i|$.

A classical example is **image classification**: given a dataset of images each labeled with a category (e.g., "cat" or "dog"), the model learns a function that maps (unseen) images to the correct label.

The key idea here is that the <u>labels are the actual solution to the learning task</u> that, in principle, a human would determine, not just a **proxy** (see further).

Unsupervised learning

Unsupervised learning (UL) is the type of learning problem in which we are given **unlabeled data** without any predefined target values assigned to them.

UL aims to find inherent relationships that are present, albeit in a hidden fashion, in the data.

Classical examples include:

- **clustering problems** such as k-means clustering, where one looks for a partition of the data points $\{x_i\}_{i\in[k]}$ where similar data points are grouped together.
- **dimensionality reduction**, where one looks for a lower-dim. representation of the given high-dimensional data which preserves some of its essential information and topology.

A practical example is **customer segmentation**: given purchase histories of users, we group customers into segments with similar buying behavior without any predefined group labels.

Semi-supervised learning

Semi-supervised learning (SSL) combines a small amount of **labeled data** with a large amount of **unlabeled data** during training.

The key assumption is that the structure revealed by the unlabeled data (e.g., clusters) can guide the model toward better generalization than the few labels alone would permit.

An example is **medical image diagnosis**: obtaining expert annotations for medical scans is expensive, so one may have only about 100 labeled scans but 1000+ unlabeled ones. SSL may leverage the unlabeled scans by *augmenting* the loss function (which, in the SL setting, would just account for the learning errors on the 100 labeled scans) with a term that penalizes inconsistent predictions when the same unlabeled input is perturbed in two different ways (a measure of **consistency**):

$$L = \underbrace{\frac{1}{|N_S|}\sum_{i=1}^{N_S} \ell\big(f_w(x_i),\, y_i\big)}_{\text{supervised loss on labeled data}} + \lambda \underbrace{\frac{1}{|N_U|}\sum_{j=1}^{N_U} d\big(f_w(\tilde{x}_j),\, f_w(\tilde{x}_j')\big)}_{\text{consistency on unlabeled data}},$$

where $\tilde{x}$ and $\tilde{x}'$ are two perturbations of x_i and λ is a **regularization hyperparameter**.

Transductive learning

Transductive learning is a setting in which, at training time, we are given both the labeled training set $\{(x_i, y_i)\}_{i \in [k]}$ and the specific unlabeled test points $\{x_j\}_{j \in J}$ for which predictions are required.

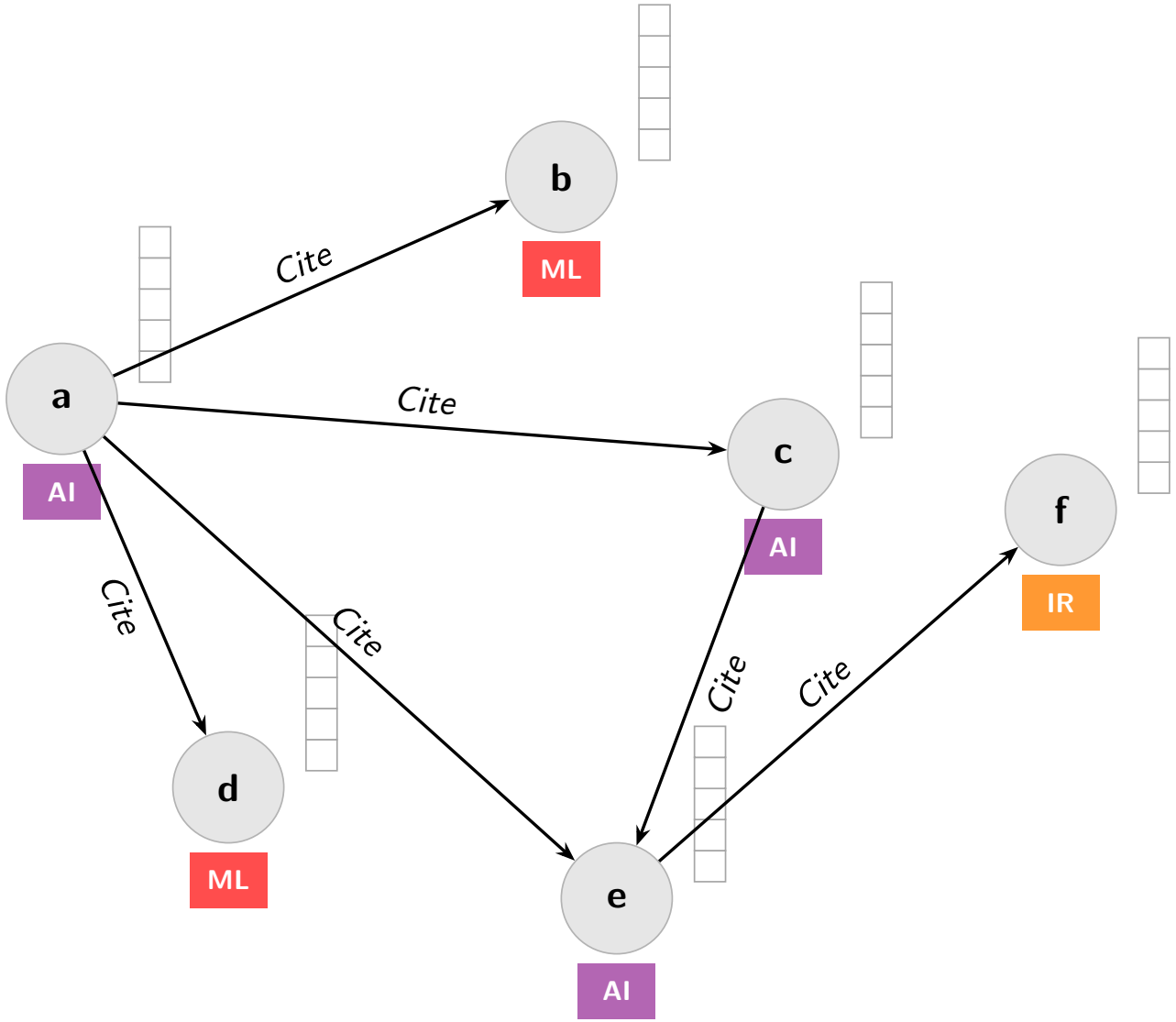
Unlike inductive (supervised) learning, the goal her eis **not** to learn a general function f_w defined on the entire input space, but rather to directly infer the labels of the given test points.

As Vapnik famously argued: "when solving a problem of interest, do not solve a more general problem as an intermediate step."

A typical example is **node classification in a social network**: given a graph where nodes are papers and edges represent citations, a small subset of papers is labeled with the scientific area they belong to (e.g., ML, AI, IR). The goal is to predict the labels of the remaining unlabeled nodes in that same graph, rather than to learn a general classifier that transfers to unseen graphs or future papers. The graph structure (who cites whom) serves as the key **inductive bias**, since connected users tend to share interests. We may talk about this more if time allows.

An example: the CiteSeer graph

Nodes = papers (3,327); edges = citations (4,732); node features = 3,703-dimensional binary bag-of-word vectors; communities (node labels) = topics (6).



Self-supervised learning

Self-supervised learning (SelfSL) is a form of learning in which the **supervision signal is (automatically) extracted from the data itself**, without human-provided labels.

One designs a **pretext** (or **proxy**) **task**, such as predicting a *masked* portion of the input or distinguishing between augmented views of the same datum.

A prominent example is the way modern (large) language models are trained (sometimes called **masked language modeling**) (as in BERT or the GPT series): given a sentence with certain tokens replaced by a `[MASK]` symbol, the model learns to predict the missing tokens, acquiring **linguistic representations** in the process.

The key understanding here is that the end-task is not filling out the blanks in a piece of text but, for the GPT models for instance, being able to talk.

Reinforcement learning

Reinforcement learning (RL) considers an **agent** that interacts sequentially with an **environment**.

At each time step t, the agent observes a state s_t, selects an action a_t according to a policy π, and receives a scalar reward r_t. The objective is to learn a policy π^* that maximizes the expected cumulative reward $\mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$, where $\gamma \in [0, 1]$ is a discount factor. No explicit input–output pairs are provided; learning is driven entirely by the **reward signal**.

A celebrated example is **AlphaGo**: the agent learns to play the board game Go by playing against itself and receiving $+1$ or -1 depending on the outcome (ultimately surpassing human-level play). It's newer iteration **AlphaZero** doesn't even know the rules of the game: everything it does is learned from the reward signal.

Strategies of learning

These are often referred to as types of "learning", but they are more like **strategies** that can be applied within any of the previous paradigms.

Transfer learning: pre-train a model on a large **source/proxy task** (e.g., LLM pre-training), then **fine-tune** it on a smaller **target task** (e.g., answering questions by following, in a supervised way, human-supplied question-answer pairs). The learned representations **transfer** (carry over) from the 1st to the 2nd task, reducing the need for target data.

Meta-learning ("learning to learn"): train a model across many related tasks so that it can adapt to a new, unseen task from very few examples (few-shot learning).

Federated learning: train a shared model across multiple decentralized devices or institutions without exchanging raw data — a privacy and infrastructure constraint, not a change in supervision.

Active learning: the learner selectively queries an oracle (e.g. a human annotator) for the labels of the most informative data points, minimizing annotation cost.

Curriculum learning: order training examples from easy to hard rather than shuffling randomly, loosely mimicking human pedagogy.

Online learning: data arrives sequentially and the model updates incrementally, as opposed to training on a fixed dataset.

Multi-task learning: train a single model on multiple related tasks simultaneously, encouraging shared representations.

4. Overfitting and the train/validate/test split

Overfitting

An **overfitting** phenomenon occurs whenever a ML model learns some of the noise that naturally affects our training data rather than managing to identify the underlying patterns.

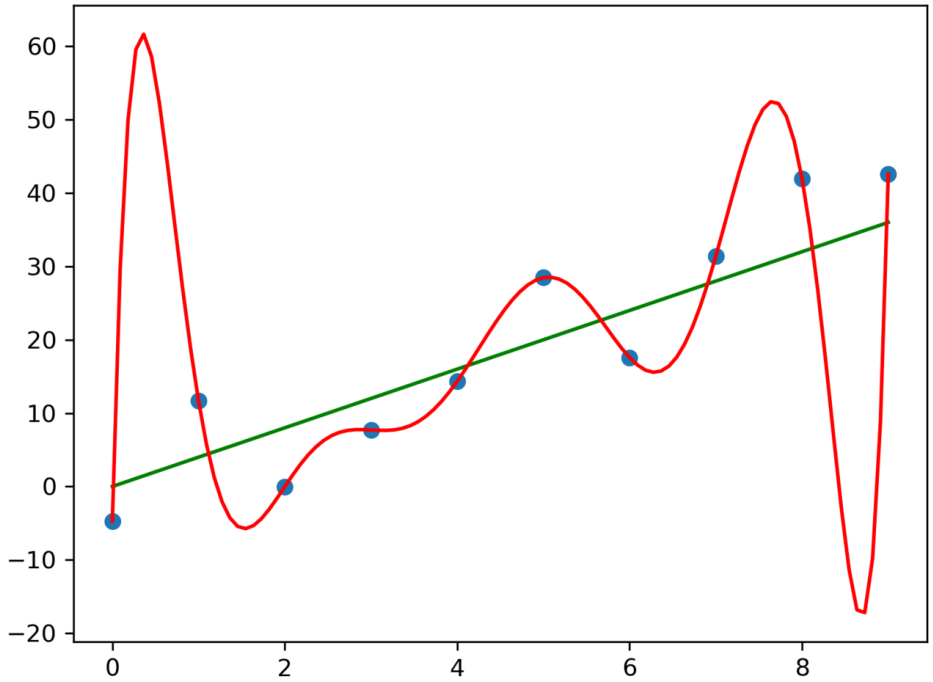
This leads to a model that performs exceptionally well on the training set but which generalizes rather poorly to unseen (i.e., **out of sample**) data.

This is not different from studying for an exam by memorizing the answers to past questions instead of understanding the concepts—when the exam questions change slightly, the student is left unprepared.

An example of overfitting

(Src: https://en.wikipedia.org/wiki/Overfitting)

The picture shows a dataset featuring noisy (roughly linear) data, fitted to a linear function (green) and a high-degree polynomial function (red).



The polynomial function perfectly fits the dataset but, if the hidden process ρ that generated it is a linear process with an additional (possibly white) noise, it is very unlikely that unseen points will be close to the red curve.

Ockham's razor

Ockham's razor (Latin: *novacula Occami*) is a parsimony principle attributed to the 14th-century English philosopher and theologian William of Ockham. It is often summarized in the Latin sentence:

> *Entia non sunt multiplicanda praeter necessitatem.*

or

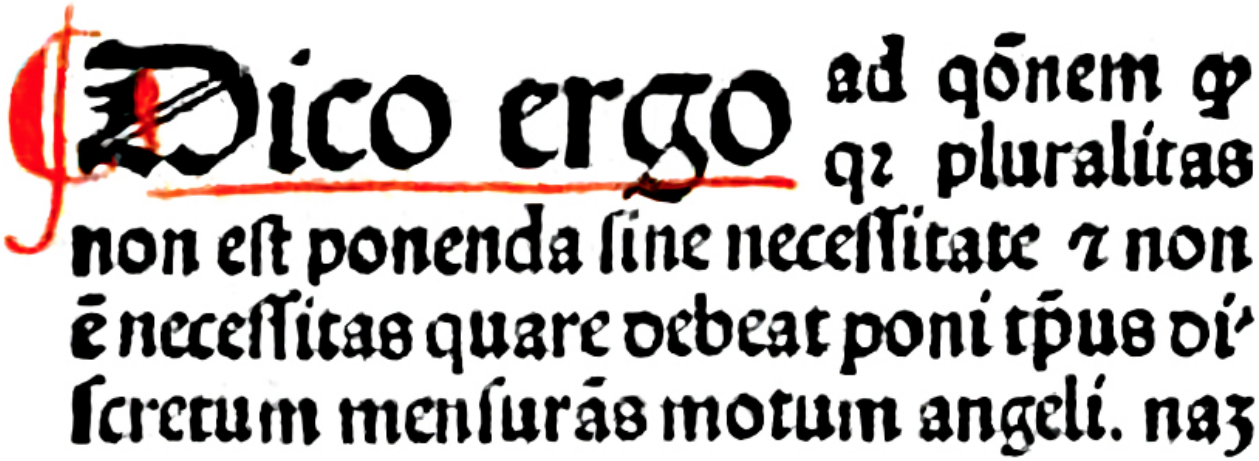
> *Entities should not be multiplied beyond necessity.*

even though these are not Ockham's exact words.

The quote recommends searching for the easiest possible explanation of a phenomenon. It's a fairly reasonable principle. Aristotle (384–322 BC) himself in the *Analytica Posteriora* wrote:

> *We may assume the superiority ceteris paribus of the demonstration which derives from fewer postulates or hypotheses.*

The origins of Ockham's razor are traceable to the works of the philosopher John Duns Scotus (1265–1308). In the book *Commentaria oxoniensia ad IV libros magistri Sententiarus*, he reports:


Dico ergo ad qōnem ꝙ
qz pluralitas
non est ponenda sine necessitate ⁊ non
ē necessitas quare debeat poni tp̄us di-
scretum mensurās motum angeli. naz

which can be transcribed as

> *Dico ergo ad quaestionem quod pluralitas non est ponenda sine necessitate [et non est necessitas quare debeat poni tepmus discretum mensuras motum angeli].*

or

> *I reply, therefore, to the question that plurality should not be posited without necessity [and there is no necessity why a discrete time should be posited regarding the motion of an angel].*

Training, validation, and test sets

To control and detect overfitting in machine learning, the dataset should not be used entirely for training.

The dataset is normally split into a **training set**, a **validation set**, and a **test set**, typically with a 70–80% split for training, a 10–15% split for validation, and a 10–15% split for testing.

- **Training set**: used to optimize the model parameters. When evaluated on this set, the loss function is called the **training loss**.
- **Validation set**: used during model development to tune hyperparameters (e.g., architecture, regularization, learning rate) and to monitor generalization. If, for a certain choice of hyperparameters, the **validation loss** is high while the training loss is low, the model is likely **overfitting**. If the training loss keeps decreasing but the validation loss stops doing so, we are likley overfitting and **early stopping** should be applied.
- **Test set**: used only after training and model selection are complete to assess the final performance of the model. Reported results in scientific papers are evaluated on this set.